

BDA Session Supplement “ 2026-09-29

In-Database Analytics: Advanced SQL Window Functions & Apache MADlib

“ Content synthesized from training knowledge (YouTube transcripts unavailable “ proxy restricted).

Topics Covered

Advanced SQL Window Functions and Greenplum Extensions for In-Database Analytics, Apache MADlib for Solving Analytics Problems in Database

Topic 1: Advanced SQL Window Functions & Greenplum Extensions

Definition & Background

SQL window functions “ formally defined in the SQL:2003 standard “ are aggregate operations that compute results over a “window” of rows relative to the current row, without collapsing the result set into a single output row per group. Unlike GROUP BY, which destroys individual row identity, OVER() preserves every row while simultaneously enriching it with partition-level or frame-level metrics.

Greenplum Database is a massively parallel processing (MPP) fork of PostgreSQL, designed specifically for analytical workloads on large datasets. It distributes data across multiple segment servers, and every query “ including window functions “ runs in parallel across all segments. A Greenplum cluster with 96 segments can execute 96 independent slices of a window function simultaneously, making it capable of real-time analytics on datasets that would cripple a single-node database.

Key Terms

Term	Definition
Window function	An SQL aggregate (RANK, SUM, LAG, etc.) that computes over a set of rows defined by an OVER() clause without collapsing the result set
OVER() clause	Defines the window: which rows are included, how they are grouped, and in what order they are processed
PARTITION BY	Divides rows into independent groups (partitions) within the window; the function resets for each new partition
ORDER BY (inside OVER)	Establishes a row ordering within each partition; required for ranking and frame-based functions like LAG and cumulative sums
Frame	The subset of rows within a partition that the function looks at; defined with ROWS BETWEEN or RANGE BETWEEN
RANK()	Assigns a rank to each row within a partition; tied rows share the same rank, and the next rank is skipped
DENSE_RANK()	Like RANK() but does not skip ranks after a tie (1, 2, 2, 3 instead of 1, 2, 2, 4)
ROW_NUMBER()	Assigns a unique sequential integer to each row in its partition, with no ties
LAG(expr, n)	Returns the value of expr from n rows before the current row in the window ordering
LEAD(expr, n)	Returns the value of expr from n rows after the current row
NTILE(n)	Divides rows in a partition into n approximately equal groups (buckets), numbered 1 to n
PERCENTILE_DISC(f)	Returns the discrete value at the f-th percentile of an ordered set (exact match from the data)

PERCENTILE_CONT(f)	Returns the continuous (interpolated) value at the f-th percentile
DISTRIBUTED BY	Greenplum-specific DDL that determines which segment server stores each row, based on a hash of the specified column(s)
Segment	An individual PostgreSQL instance within a Greenplum cluster; each stores a partition of the data and executes queries in parallel
Data skew	Uneven distribution of rows across Greenplum segments; causes some segments to do disproportionately more work, slowing the entire query
MPP (Massively Parallel Processing)	An architecture where many processors operate on different data simultaneously, each with its own memory and storage

Deep Dive “ How It Works

The anatomy of a window function call:

```
function_name(arguments)
OVER (
    PARTITION BY partition_columns
    ORDER BY ordering_columns
    ROWS BETWEEN frame_start AND frame_end
)
```

Every clause is optional except the `OVER()` itself. An empty `OVER()` “ `SUM(revenue) OVER ()` ” computes the grand total for every row without any partitioning or ordering.

How Greenplum executes window functions:

Greenplum™s query planner evaluates window functions in a distinct phase “ after `WHERE` and `JOIN` but before `HAVING` and outer `SELECT` expressions. The execution sequence is:

1. **Redistribute** “ rows are reshuffled across segments so that all rows with the same `PARTITION BY` value land on the same segment. If `DISTRIBUTED BY` matches `PARTITION BY`, this shuffle is eliminated entirely “ a major performance win.
2. **Sort** “ within each segment, rows are sorted by the `ORDER BY` clause.
3. **Scan** “ for each row, the database evaluates the window function against the frame of rows defined by `ROWS BETWEEN`.
4. **Return** “ each row is emitted with the computed window value appended; the result set size is identical to the input.

Multiple window functions with the same `PARTITION BY` and `ORDER BY` clauses are batched into a single pass over the data. The query planner recognises identical window specifications and avoids redundant sorting.

Frame clauses in depth:

The frame clause restricts which rows within the partition participate in the calculation:

- `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` “ cumulative aggregate (running sum)
- `ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING` “ 5-row rolling average
- `ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING` “ suffix aggregate (e.g., remaining balance)
- `RANGE BETWEEN INTERVAL '7 days' PRECEDING AND CURRENT ROW` “ time-based rolling window (requires `ORDER BY` on a date/timestamp column)

Greenplum-specific considerations:

Distribution key selection is the single most impactful architectural decision for analytical performance. If the `PARTITION BY` column in a heavily used window query is not the distribution key, Greenplum must redistribute data across the network for every query execution “ this is called a “motion” and can dominate query runtime. For a table of restaurant orders where most analytics are city-level, `DISTRIBUTED BY (city)` ensures city-partitioned window functions run locally on each segment.

Partition pruning “ Greenplum™s planner eliminates segments that provably hold no matching rows. A query with `WHERE city = 'Mumbai'` against a range-partitioned table only touches the Mumbai partition.

Common Pitfalls & Anti-patterns

- 1. Omitting ORDER BY when it matters** `SUM()` `OVER (PARTITION BY x)` without `ORDER BY` computes the total for the entire partition – identical for every row. If you want a *running* sum, you must add `ORDER BY date`. Many developers write the first form expecting the second behaviour.
- 2. Confusing RANK() and ROW_NUMBER()** `ROW_NUMBER()` is deterministic only if the `ORDER BY` is a unique column. On a non-unique column (e.g., `ORDER BY city`), `ROW_NUMBER()` assigns arbitrary but stable numbers to tied rows – which is fine if you just want a numbered list, but misleading if you want to identify actual ties. Use `RANK()` or `DENSE_RANK()` when ties have semantic meaning.
- 3. Poor distribution key causing full network reshuffle** If your most-queried `PARTITION BY` column is not your `DISTRIBUTED BY` column, every window function query triggers a redistribute motion across the entire cluster. Profile with `EXPLAIN ANALYZE` and look for “Redistribute Motion” nodes. When you see one under a Window node, consider changing the distribution key or creating a projection (materialized view) with the right distribution.
- 4. Using window functions where a subquery is clearer** `FILTER (WHERE ...)` inside a window function can be replaced with a CTE or subquery that is often more readable and equally performant. Over-engineering with window functions makes queries hard to maintain.
- 5. Frame clause mis-specification for rolling aggregates** `RANGE BETWEEN 6 PRECEDING AND CURRENT ROW` on an integer `ORDER BY` column means “rows whose value is within 6 of the current row” – not “the 6 preceding rows in sort order.” Use `ROWS BETWEEN` when you want a fixed count of physical rows.

Additional Resources

- [PostgreSQL Window Functions Tutorial](#) – the canonical reference; applies fully to Greenplum
- [Greenplum Best Practices Guide – Table Distribution](#)
- [Use the Index, Luke – Window Functions](#) – excellent visual explanation of frame semantics

Topic 2: Apache MADlib – In-Database Machine Learning

Definition & Background

Apache MADlib (MAD = Magnetic, Agile, Deep) is an open-source library of machine-learning and statistical algorithms implemented as PostgreSQL/Greenplum user-defined functions. Developed originally at UC Berkeley in collaboration with Pivotal (now VMware), MADlib was designed around a simple principle: **move the algorithm to the data, not the data to the algorithm.**

Traditional ML pipelines follow an extract-transform-load pattern: pull training data from the database into a data-science environment (Python, R, Spark), train a model, serialize it (pickle, PMML, ONNX), deploy it to a serving layer, and periodically retrain. Each step introduces latency, infrastructure cost, and potential for data drift between training and serving distributions.

MADlib eliminates the extract step by running algorithms natively in SQL. A logistic regression call – `madlib.logregr_train(...)` – is syntactically identical to a `SUM()` or `RANK()` call. Greenplum’s query executor treats it the same way: it fans out to all segments in parallel, each segment trains on its local data slice, and the results are aggregated to produce global model coefficients. The final model is written to a regular database table, which can be queried, versioned, and joined against like any other table.

Key Terms

Term	Definition
In-database ML	Machine learning where algorithms execute inside the database engine, with no data movement to an external ML platform
madlib.linregr_train()	Trains an ordinary least-squares linear regression model; outputs coefficients, standard errors, R^2 , p-values
madlib.logregr_train()	Trains a logistic regression classifier for binary outcomes; uses iterative reweighted least squares (IRLS) or conjugate gradient

madlib.logregr_predict()	Scores new rows using a trained logistic regression model; returns class label (0 or 1)
madlib.logregr_predict_prob()	Returns predicted probability of the positive class for each row
madlib.kmeans()	Groups rows into k clusters by minimising within-cluster sum of squared distances; supports multiple distance metrics
madlib.svm_train()	Trains a support vector machine for classification or regression; supports linear, polynomial, and RBF kernels
madlib.naive_bayes_train()	Trains a Naïve Bayes classifier; very fast, good baseline for text classification
madlib.arima_train()	Fits an ARIMA time-series model to a sequence of observations
Parallel gradient descent	MADlib™'s optimisers distribute weight updates across Greenplum segments; each segment computes a local gradient on its data, and the master node aggregates them before the next iteration
IRLS (Iteratively Reweighted Least Squares)	Default optimiser for logistic regression in MADlib; generally converges in 10–30 iterations for well-conditioned problems
Confusion matrix	Table comparing predicted class labels to actual labels; rows = predicted, columns = actual
AUC (Area Under the ROC Curve)	Model quality metric ranging 0–1; 0.5 = random, 1.0 = perfect; 0.75+ is generally useful for production
Feature array	MADlib expects features as a PostgreSQL array column (ARRAY[col1, col2, ...]); create this with a SELECT before training
Output table	MADlib writes model parameters (coefficients, metadata) to a user-specified table; query it to inspect the model

Deep Dive – How It Works

The MADlib execution model:

MADlib algorithms are implemented as user-defined aggregate functions (UDAFs) in PostgreSQL/C. A UDAF has three components:

1. **Transition function** – called once per row; updates a running state (e.g., accumulates gradient contributions)
2. **Merge function** – combines two partial states; used by Greenplum to merge results from different segments
3. **Final function** – converts the accumulated state into the final output (model coefficients)

Greenplum™'s parallel executor invokes the transition function on each segment independently, then uses the merge function to combine segment results on the master node. For logistic regression, this means each segment computes a partial gradient over its rows, the segments' gradients are merged (summed), and the master applies the parameter update. This continues for up to `max_iterations` passes – each pass is one epoch over the full dataset.

Training a logistic regression model – step by step:

```
-- Step 1: Create a feature array column (MADlib expects arrays)
CREATE TABLE customer_features_array AS
SELECT customer_id,
       ARRAY[tenure_months::float,
             monthly_spend::float,
             support_tickets::float,
             login_freq_30d::float] AS features,
       churned
FROM   customer_features;

-- Step 2: Train the model
SELECT madlib.logregr_train(
    'customer_features_array', -- source table
    'churn_model',            -- output table (created automatically)
    'churned',                -- dependent variable (0 or 1)
    'features',               -- independent variable (array column)
    NULL,                    -- grouping column (train one model for all groups)
    20,                      -- max_iterations
```

```

    'irls'                                -- optimiser: 'irls' or 'cg'
);

-- Step 3: Inspect the trained model
SELECT unnest(ARRAY['tenure_months','monthly_spend',
                    'support_tickets','login_freq_30d']) AS feature,
       unnest(coef)      AS coefficient,
       unnest(std_err)   AS std_error,
       unnest(p_values) AS p_value,
       log_likelihood,
       num_iterations
FROM   churn_model;

-- Step 4: Score the live customer table
SELECT c.customer_id,
       madlib.logregr_predict(m.coef, c.features)      AS predicted_churn,
       madlib.logregr_predict_prob(m.coef, c.features) AS churn_probability
FROM   customer_features_array c
CROSS JOIN churn_model m
ORDER BY churn_probability DESC;

```

The `churn_model` table is an ordinary SQL table. It can be version-controlled by creating a new table per retraining run (`churn_model_2026_09_29`), queried to audit coefficient drift over time, and joined against a scoring query that runs on a schedule.

k-means clustering in MADlib:

```

-- Cluster customers into 4 segments by spend, engagement, and tenure
SELECT madlib.kmeans(
    'customer_features_array',          -- source table
    'ARRAY[monthly_spend::float,
          login_freq_30d::float,
          tenure_months::float]',      -- feature expression (not a column)
    4,                                  -- k = 4 clusters
    'madlib.dist_norm2',                -- Euclidean distance
    'madlib.avg',                       -- centroid averaging method
    20,                                 -- max iterations
    0.001                               -- convergence threshold
);

```

MADlib returns a table with cluster assignments and centroid coordinates. You join it back against the source table to label each customer with their segment.

Model evaluation functions:

MADlib provides dedicated evaluation functions that also run in-database:

```

-- Cross-validation (5-fold)
SELECT madlib.cross_validation_general(
    'madlib.logregr_train', ..., 5
);

-- ROC curve / AUC
SELECT madlib.area_under_roc('predictions', 'churned', 'churn_probability');

```

Common Pitfalls & Anti-patterns

1. Not scaling features before clustering or SVM MADlib's k-means and SVM are distance-based. Features with larger numeric ranges (e.g., monthly spend in thousands vs. a 0-10 engagement score) dominate the distance calculation. Always normalise with $(\text{value} - \text{mean}) / \text{stddev}$ either in a preprocessing CTE or using MADlib's `madlib.normalize()`.

2. Training on unbalanced classes without resampling If only 3% of customers churn, a naive logistic regression can achieve 97% accuracy by predicting "no churn" for everyone. Oversample the minority class, undersample the majority, or use `class_weight` in `madlib.logregr_train()` to penalise misclassification of the positive class.

3. Using CROSS JOIN for scoring without a WHERE clause The canonical scoring pattern `FROM customer_features CROSS JOIN churn_model` works because `churn_model` has exactly one row. If you accidentally train with a grouping column that creates multiple rows in the output table, the `CROSS JOIN` multiplies the scoring table – verify `SELECT COUNT(*) FROM churn_model` returns 1 before running

production scoring.

4. Ignoring num_iterations in the output If `num_iterations` equals `max_iterations`, the model did not converge. Either increase `max_iterations`, normalise features, or check for multicollinearity. A non-converged model has unstable coefficients.

5. Treating MADlib as a replacement for all ML infrastructure MADlib excels at classical statistical ML (linear/logistic regression, k-means, SVM, Naive Bayes). For deep learning, large embedding models, or real-time serving at millisecond latency, a dedicated ML platform (SageMaker, Vertex AI) is the right choice. MADlib and external ML are complementary – use MADlib for bulk batch scoring in the warehouse, external platforms for low-latency serving.

Additional Resources

- [Apache MADlib User Documentation](#) – algorithm reference with SQL examples for every function
- [MADlib GitHub Repository](#) – source, release notes, worked Jupyter notebooks
- [Greenplum + MADlib Reference Architecture \(VMware\)](#) – enterprise deployment guide
- [In-Database Analytics with MADlib \(Coursera / Pivotal\)](#) – free course with hands-on MADlib labs

Synthesis: Connecting the Topics

Both topics today share a single underlying philosophy: **keep computation where the data lives.**

Advanced SQL window functions let analysts write calculations that used to require a Python or Spark job – ranking, rolling averages, period-over-period comparisons – directly in the database layer. Greenplum’s MPP engine makes those calculations scale linearly with cluster size without any code change.

Apache MADlib extends that philosophy into machine learning. Just as `RANK() OVER (...)` eliminates the need to export data for ranking, `madlib.logregr_train()` eliminates the need to export data for model training. The model lives in the database; scoring is a `SELECT`; the feedback loop from data to insight shrinks from days to minutes.

Together, these two capabilities mean that a Greenplum database cluster can serve as an end-to-end analytical and ML platform – ingesting raw data, transforming it with window functions, training models on it with MADlib, scoring new data in real time, and surfacing results through standard SQL – without a single byte leaving the warehouse.

The practical implication: in regulated industries like financial services or healthcare, where data governance requires all processing to happen within a certified environment, this matters enormously. Moving data to an external ML platform requires replicating access controls, audit trails, and encryption at rest. Keeping it in Greenplum means one environment to govern.

References

– Content synthesized from training knowledge – YouTube transcripts unavailable (proxy blocked during generation).

Videos (watch independently)

1. [Mastering Window Functions in SQL for Data Analysis: 1 Hour Mini-Course](#)
2. [The Ultimate SQL Window Functions Course | Complete End-to-End Guide](#)
3. [Greenplum SQL Scripts for Data Analysts](#)
4. [Greenplum Database – Massively Parallel PostgreSQL for Analytics](#)
5. [Machine Learning on Greenplum with Apache MADlib](#)
6. [LIVESTREAM: Learn about ML on Greenplum with Apache MADlib](#)
7. [Classification Models in Greenplum using Apache MADlib](#)
8. [Parallel Postgres: Gaming MAD skills with Apache MADlib and Greenplum](#)

Official Documentation

- [Apache MADlib Docs](#)
- [PostgreSQL Window Functions](#)
- [Greenplum Database Documentation](#)
- [dplyr Window Functions Vignette](#)
- [Use the Index, Luke – Window Functions](#)